

Qt Designer

Qt Designer

Кроссплатформенная свободная среда для разработки графических интерфейсов (GUI) программ использующих библиотеку Qt. Входит в состав Qt framework.

Виджеты

Любой GUI в Qt рассматривается как совокупность виджетов.

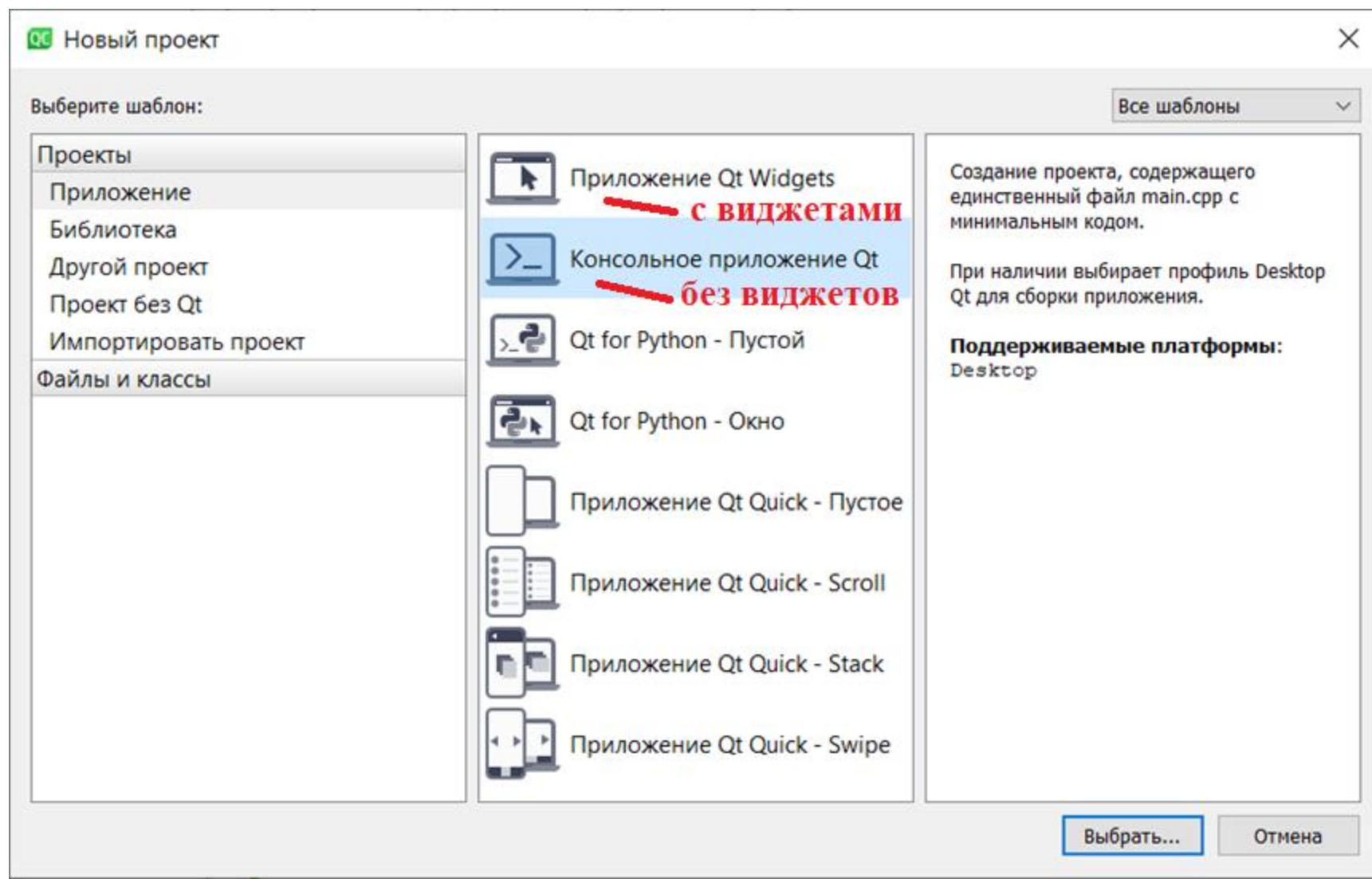
Окно приложения – виджет.

Надпись (label) – виджет.

Кнопка – виджет.

При создании GUI задача разработчика – задать правильную иерархию виджетов и их компоновку (чтобы приложение адекватно реагировало на изменение размеров окна)

Hello, Qt Designer!



Hello, Qt Designer!

  Приложение Qt Widgets

Размещение

Система сборки

 Подробнее

Комплекты

Итог

Информация о классе

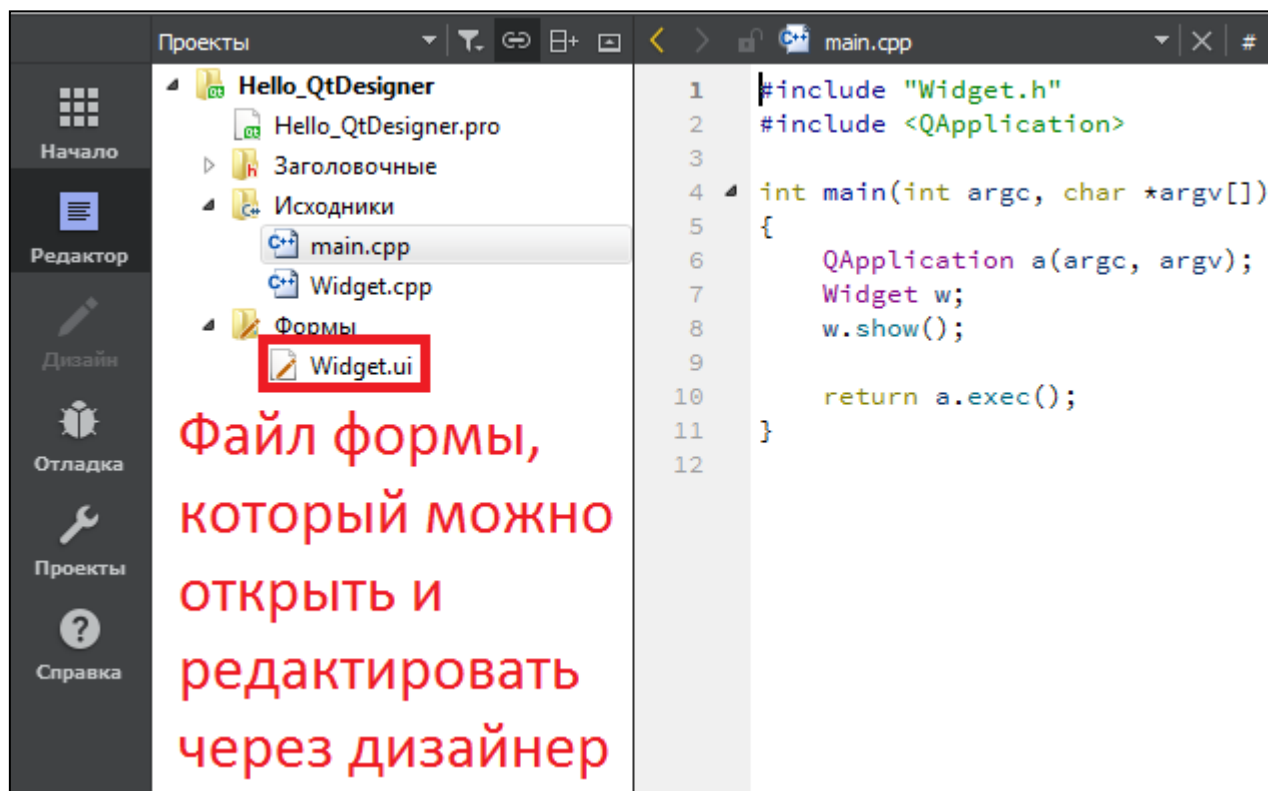
Укажите базовую информацию о классах, для которых желаете создать шаблоны файлов исходных текстов.

Имя класса:	Widget
Базовый класс:	QWidget QMainWindow QWidget QDialog
Заголовочный файл:	Widget.h
Файл исходных текстов:	Widget.cpp
	☒ Создать форму
Файл формы:	Widget.ui

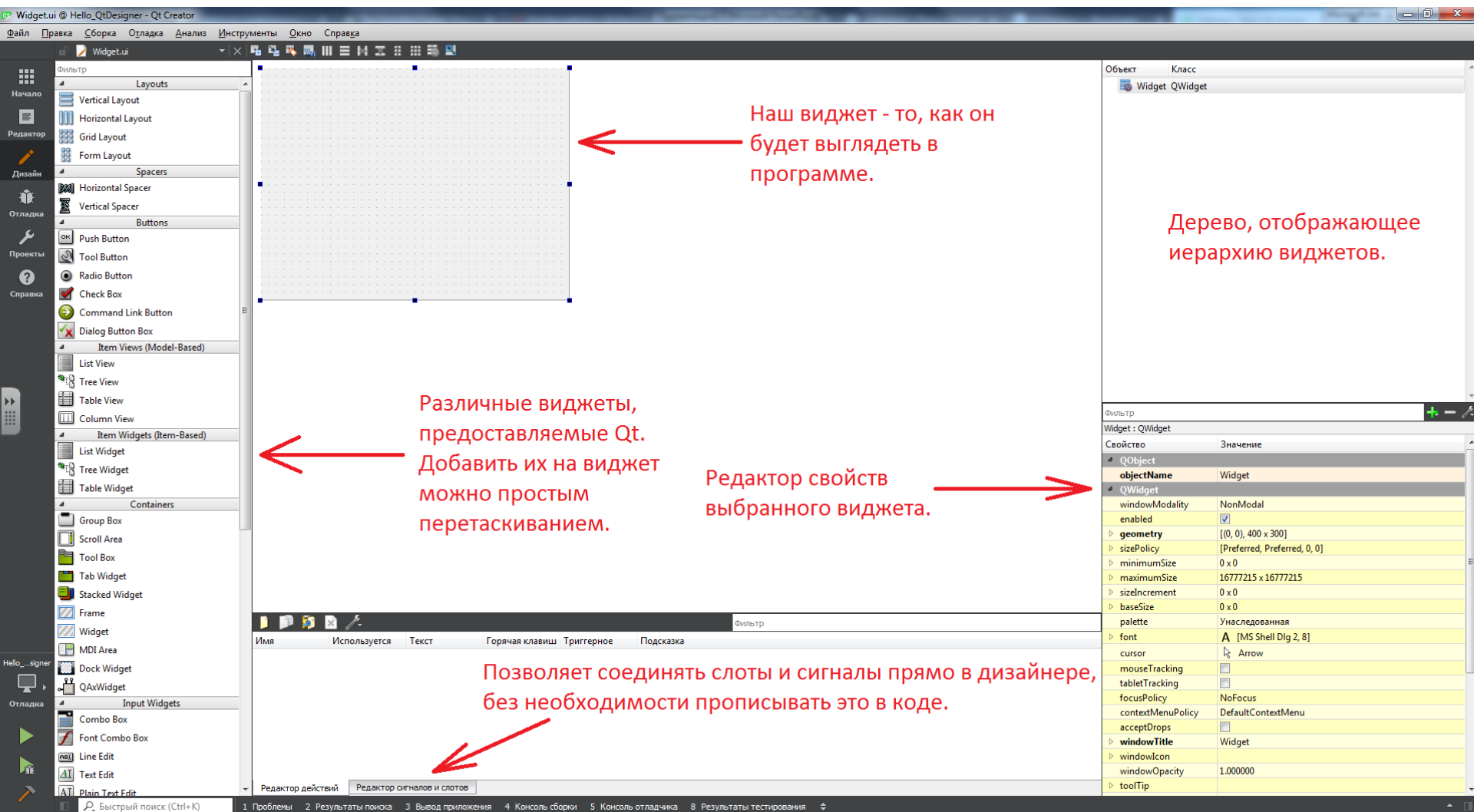
Далее

Отмена

Hello, Qt Designer!

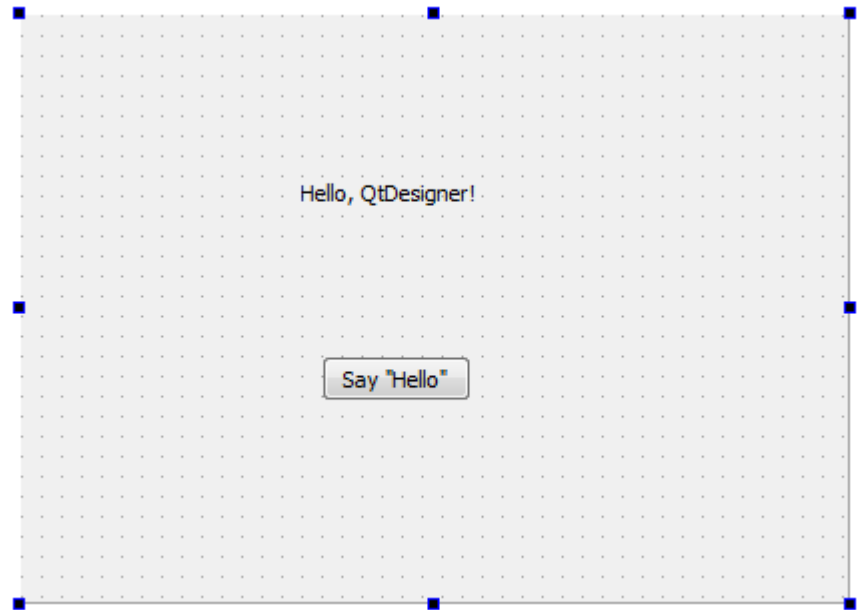


Hello, Qt Designer!



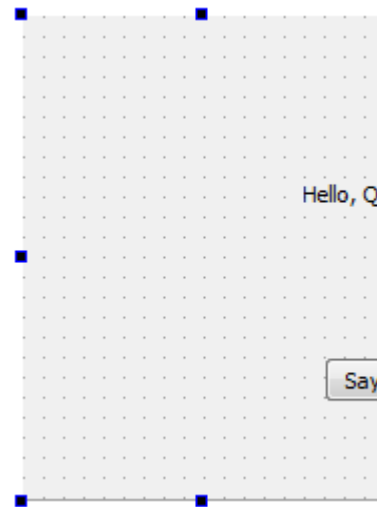
Hello, Qt Designer!

- Добавим кнопку (PushButton) и надпись (Label).
- Изменить стандартные надписи можно через редактор свойств или двойным кликом ЛКМ.



Hello, Qt Designer!

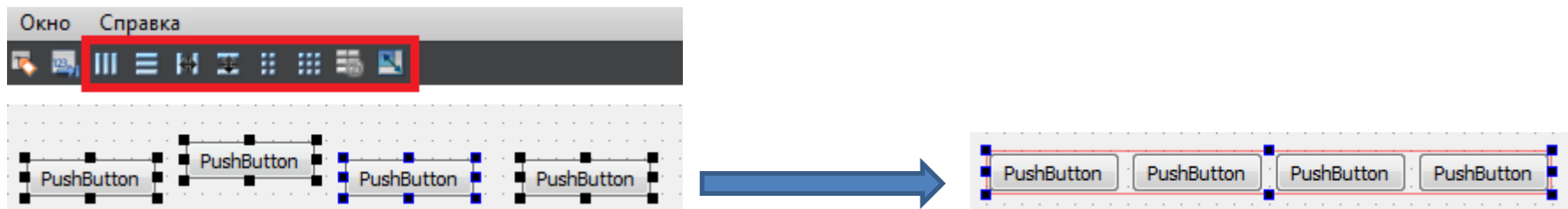
Сейчас при изменении окна дочерние элементы (кнопка и надпись) не смещаются. В итоге пользователь сможет сделать что-нибудь вроде этого:



Для решения подобных проблем в Qt существуют компоновщики.

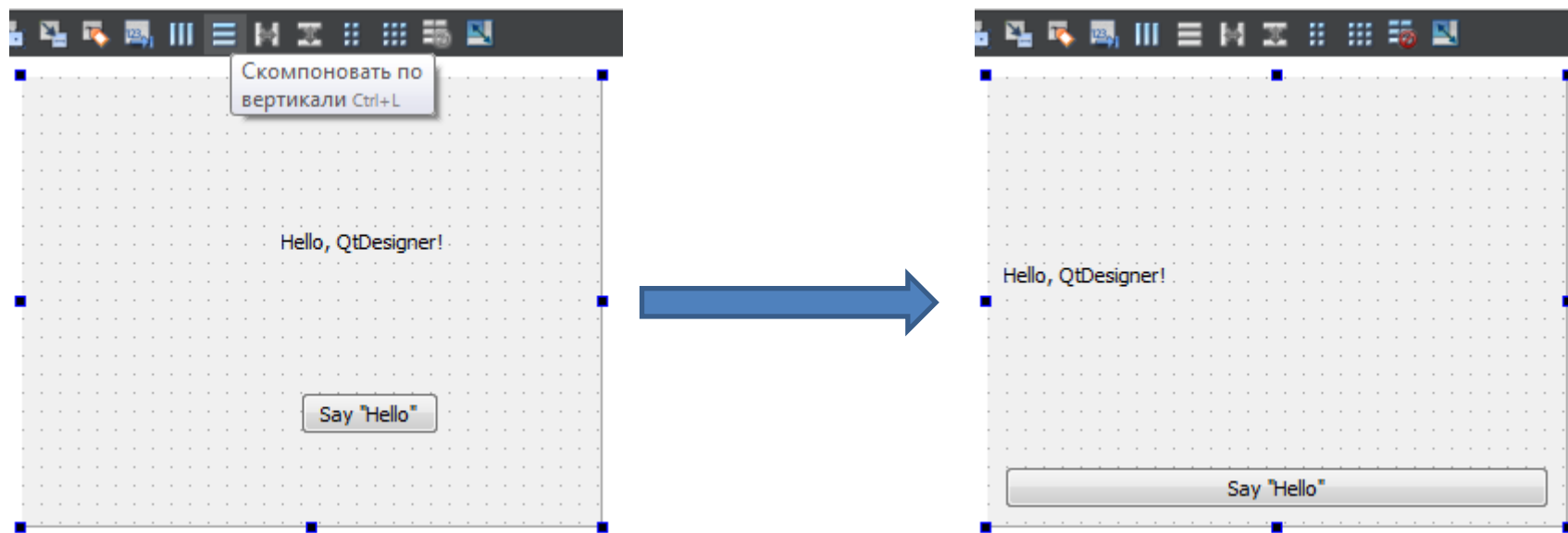
Компоновщики

Назначение компоновщиков – группировка элементов (по горизонтали, вертикали, сетке и т.п.), чтобы при масштабировании окна приложения взаимное расположение всех виджетов было корректным.



Hello, Qt Designer!

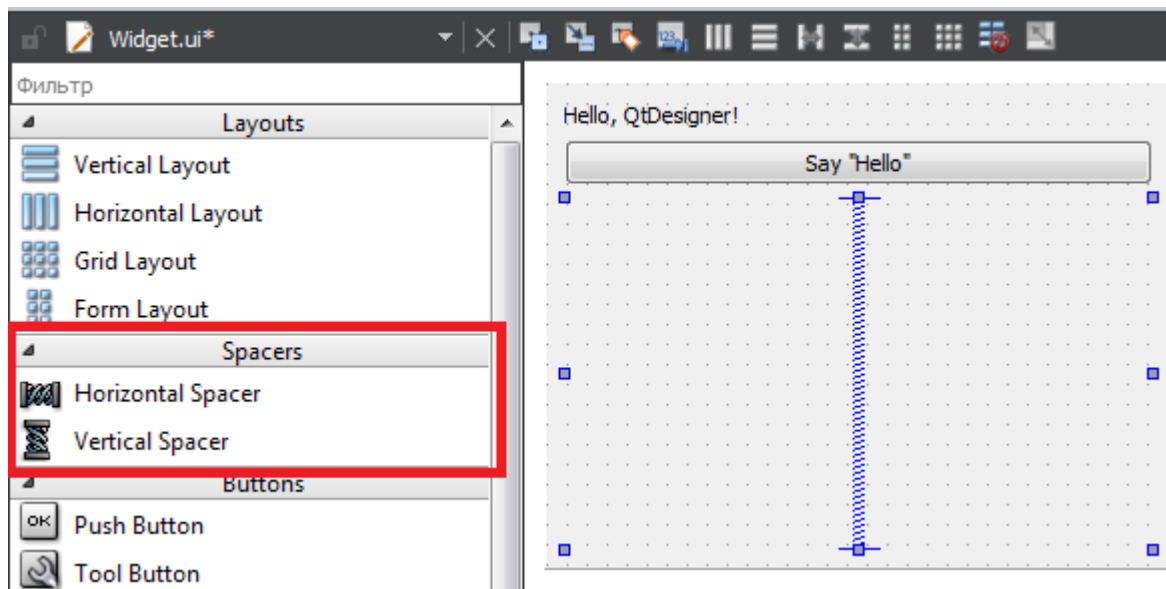
В нашем случае достаточно будет задать один КОМПОНОВЩИК – главного окна:



Обратите внимание, что выделен главный виджет, а не кнопка и надпись.

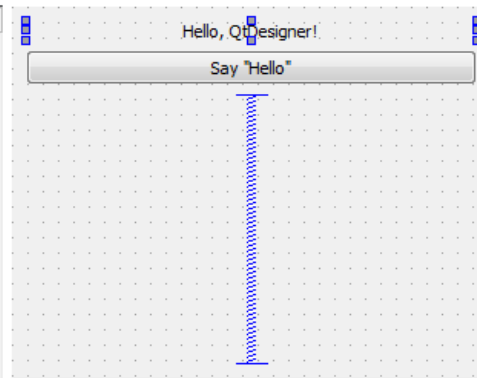
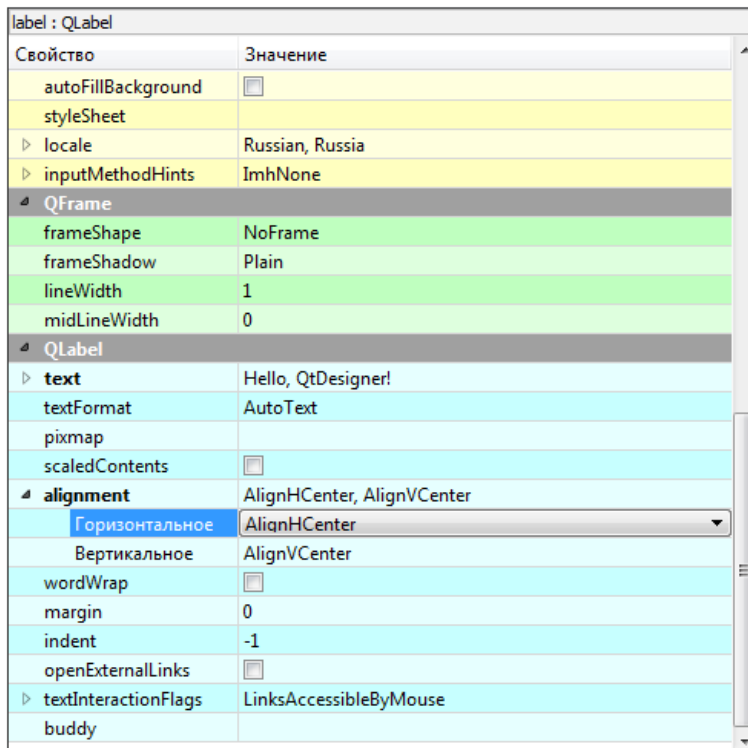
Hello, Qt Designer!

Если вам нужно, чтобы при растягивании окна виджеты были сгруппированы в какой-то одной его части, вам помогут Horizontal Spacer и Vertical Spacer:



Hello, Qt Designer!

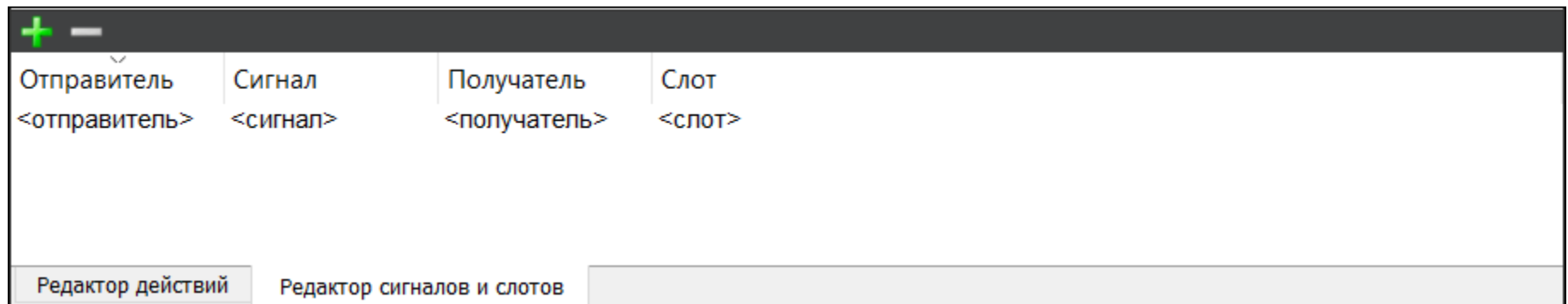
Теперь немного отредактируем нашу надпись с помощью редактора свойств: сделаем выравнивание текста по центру.



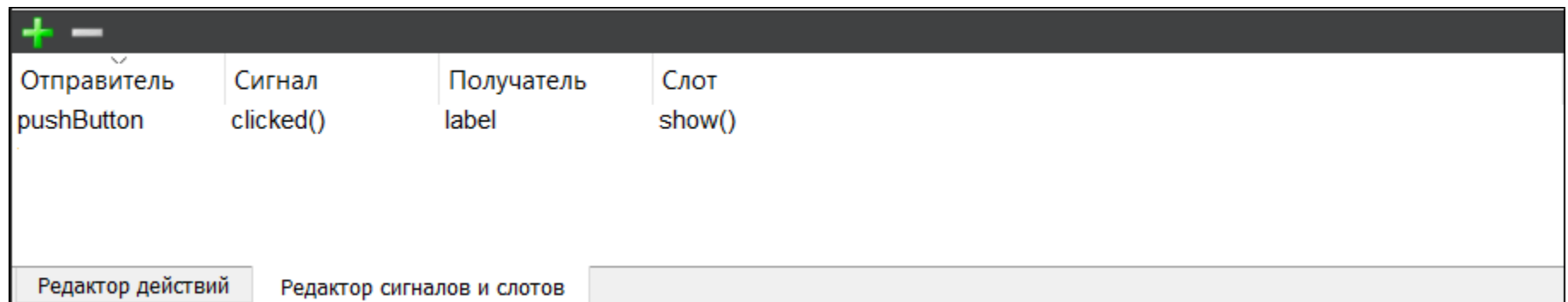
Можете отредактировать что-нибудь ещё, по своему вкусу.

Редактор сигналов и слотов

Интерфейс редактора сигналов и слотов
очень простой:

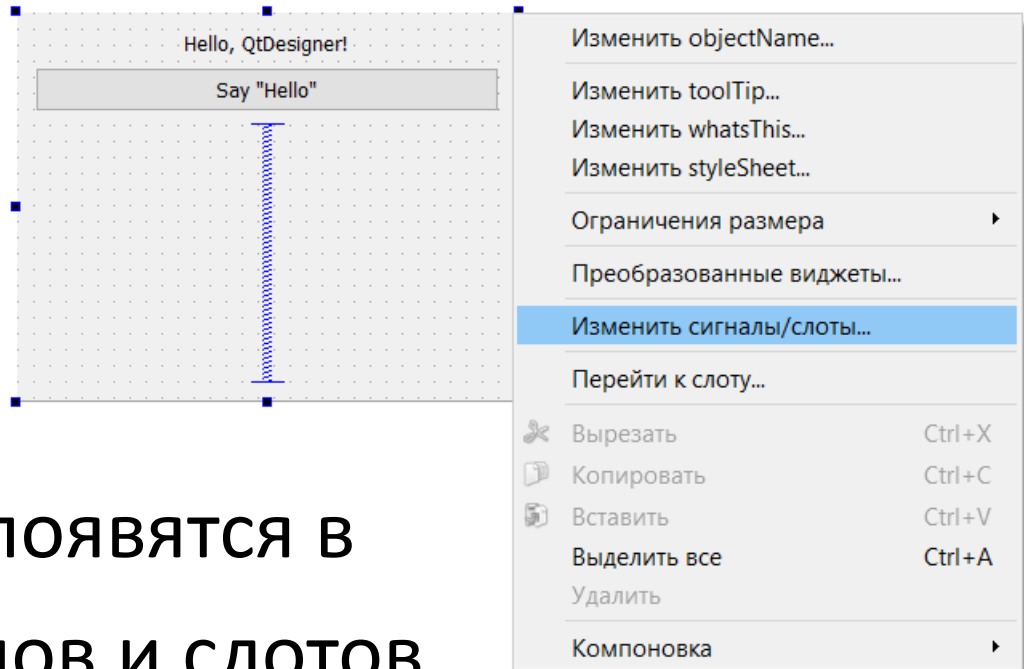


Сделаем следующую привязку:



Редактор сигналов и слотов

Если вы хотите связать свои слоты/сигналы через дизайнер, вам нужно будет добавить их своему виджету:



После этого они появятся в Редакторе сигналов и слотов.

Hello, Qt Designer!

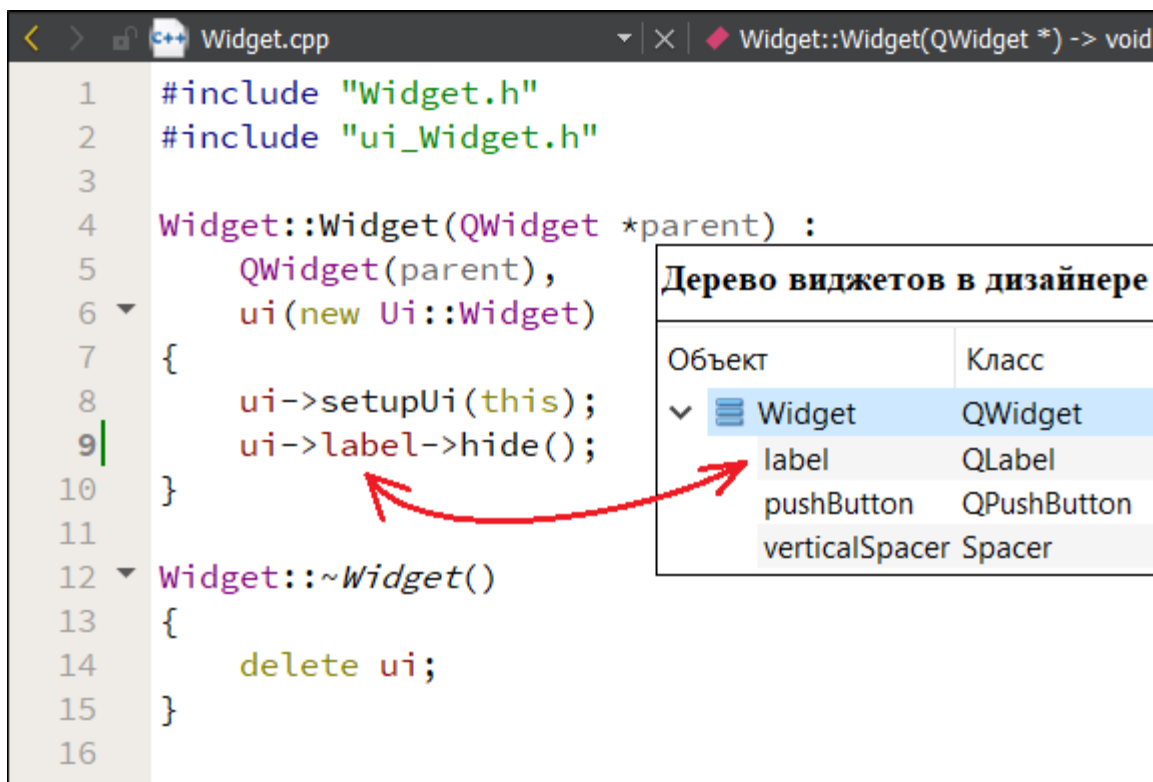
Для того, чтобы обратиться к дочерним виджетам, добавленным в дизайнера, через код, используется специальный указатель **ui**.

Посмотреть/изменить названия виджетов можно через:

- дерево виджетов (в правом верхнем углу);
- редактор свойств выбранного виджета;
- контекстное меню (ПКМ по виджету -> изменить `objectName`)

Hello, Qt Designer!

Спрячем нашу надпись в конструкторе виджета. А при нажатии на кнопку она вновь появится (см. редактор сигналов и слотов).



The screenshot displays the Qt Creator IDE. The main editor shows the `Widget.cpp` file with the following code:

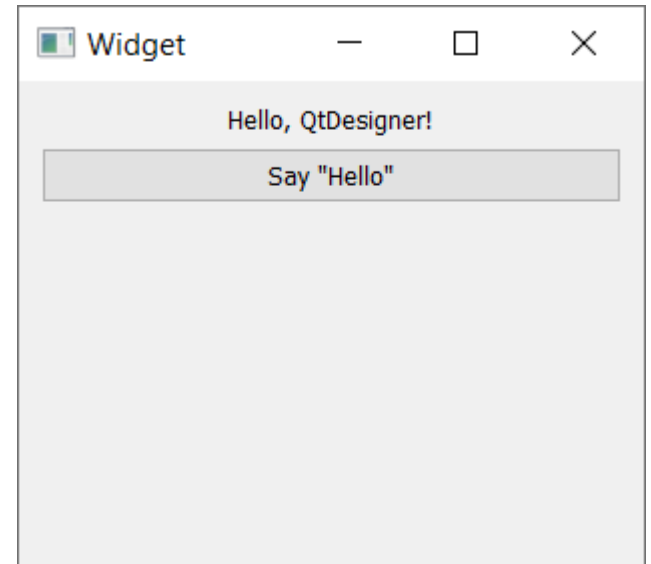
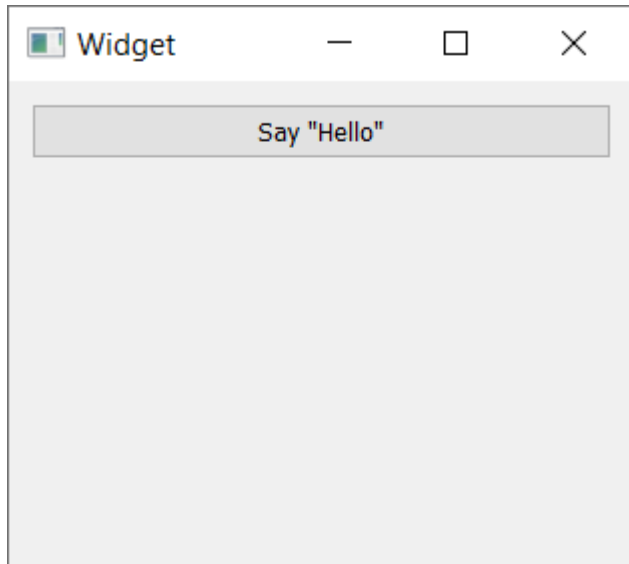
```
1  #include "Widget.h"
2  #include "ui_Widget.h"
3
4  Widget::Widget(QWidget *parent) :
5      QWidget(parent),
6      ui(new Ui::Widget)
7  {
8      ui->setupUi(this);
9      ui->label->hide();
10 }
11
12 Widget::~Widget()
13 {
14     delete ui;
15 }
16
```

On the right side, the "Дерево виджетов в дизайнере" (Widget Tree in Designer) panel is visible. It contains a table with two columns: "Объект" (Object) and "Класс" (Class). The tree structure is as follows:

Объект	Класс
Widget	QWidget
label	QLabel
pushButton	QPushButton
verticalSpacer	Spacer

A red arrow points from the `ui->label->hide();` line in the code to the `label` object in the widget tree.

Hello, Qt Designer!



Небольшое задание

- Подумайте: как сделать так, чтобы при первом нажатии на кнопку надпись появлялась, а при втором – исчезала?